Building With Peregrine Commit

A Study Guide to Our Monorepo, Design System, and Engineering Systems

Peregrine Commit Engineering

July 2026

Contents

Welcome

This guide exists so that anyone joining Peregrine Commit — whether you're a junior developer writing your first component, a senior engineer touching the build pipeline for the first time, or a product designer handing off a Figma file — can understand *how the pieces fit together* without having to reverse-engineer it from scratch.

Peregrine Commit is a **Turborepo/pnpm monorepo** that houses a marketing website, a small back-end API, and — at its heart — a from-scratch **Atomic Design component library** whose visual language (colors, spacing, type, motion) is generated from a **two-layer design token system**. That token system is not an implementation detail; it *is* the product design system, and it is meant to be read, understood, and extended by designers and engineers together.

This guide is organized so each chapter first explains the general concept (for anyone new to that idea), then shows exactly how Peregrine Commit implements it, with real file paths and real code. You do not need to read it front to back. Skim the table of contents, jump to what's relevant to your work this week, and come back to the rest as you need it.

Throughout the guide you'll see two kinds of boxes. A rust-colored box marked **"For designers"**, **"For junior devs"**, or **"For senior devs"** calls out something specific to that audience. A teal **key fact** box highlights a rule worth remembering.

> **The one-sentence version of this whole repo:** raw design values live in `token-factory`, get turned into CSS variables and Storybook-documented components in `peregrine-commit-ui`, and get consumed by `apps/web` (the marketing site) and indirectly supported by `apps/api` (the backend) — all built, linted, type-checked, and tested by Turborepo in dependency order.

Chapter 1 --- The Big Picture

What Peregrine Commit is

Peregrine Commit is two things bolted together: a **marketing site + client work platform** (the thing visitors actually see) and a **design system** that the site — and future client projects — are built from. The repository is a monorepo, meaning both of those things, plus all their shared tooling, live in one Git repository instead of being split across separate repos.

How the workspace is laid out

The repository root is a **pnpm workspace**. `pnpm-workspace.yaml` declares that anything under `apps/*` or `packages/*` is a workspace member, and every workspace member gets its own `package.json`, its own dependencies, and its own scripts, while sharing one `node_modules` install and one lockfile at the root.

There are two `apps/` (things that get deployed — a website and an API service) and four `packages/` (things that get consumed by other packages — a component library, a token library, and two shared tooling configs).

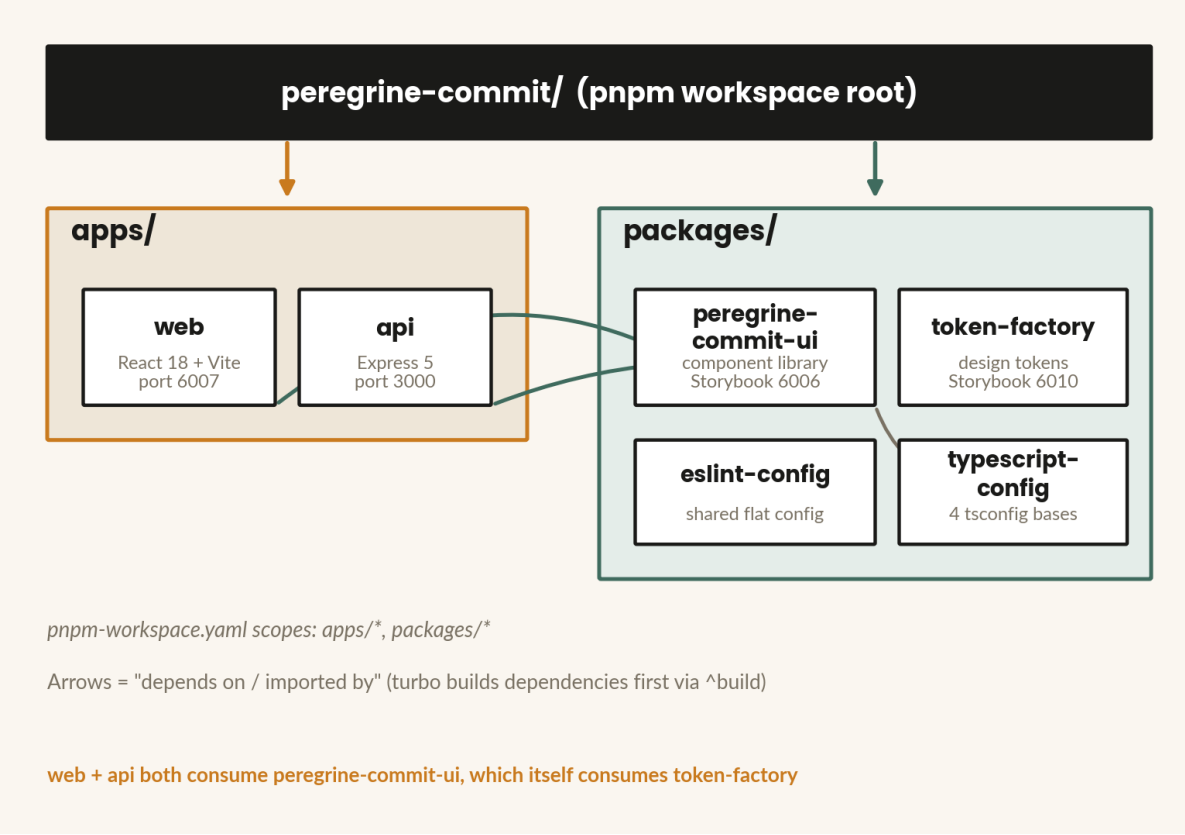

Figure 1: The monorepo workspace: apps consume packages, and packages consume each other

For junior devs

If you've only worked in single-repo projects before, the mental model to hold onto is:

> `packages/` **are libraries**, `apps/` **are products**. You'll spend most early tasks either adding a component to `peregrine-commit-ui` or wiring an existing component into an `apps/web` section — rarely inventing something from nothing.

The dependency chain, spelled out

`apps/web` and `apps/api` both depend on `@peregrine-commit/ui` (the component library). `@peregrine-commit/ui` in turn depends on `@peregrine-commit/token-factory` (the design tokens). Nothing in the repo depends on `apps/web` or `apps/api` — they're leaves in the dependency tree, which is exactly what you'd expect from deployable applications. Every package additionally pulls in `@peregrine-commit/eslint-config` and `@peregrine-commit/typescript-config` as dev-time tooling dependencies, which is why those two show up connected to everything even though they produce no runtime code.

Chapter 2 --- Monorepos, pnpm, and Turborepo

Why a monorepo at all

A monorepo is simply a single repository holding multiple, independently-versionable packages, as opposed to a "polyrepo" world where the component library, the API, and the website would each live in their own Git repository. The appeal is straightforward: a designer changing a token value and an engineer consuming that token in a component can make and review both changes in a single pull request, with the compiler catching any mismatch immediately, instead of coordinating a release across three repositories.

The tradeoff is that a monorepo needs tooling to avoid becoming slow and tangled as it grows — which is exactly the job **pnpm** and **Turborepo** split between them.

pnpm: how packages link to each other

pnpm is our package manager. Its distinguishing feature versus npm or Yarn Classic is a **content-addressable store**: every version of every package is stored once on disk, and every project's `node_modules` is built from hard links and symlinks into that store, rather than nested copies. That makes installs fast and disk-cheap, but its more important property for us is **workspaces**: pnpm understands that `@peregrine-commit/ui` inside this repo should resolve to the local `packages/peregrine-commit-ui` source (declared as `workspace:*` in `package.json`) rather than a published npm version, so a change to the component library is instantly visible to anything that imports it, with no publish step.

Turborepo: deciding what to run, and in what order

Where pnpm decides *how packages link*, **Turborepo** decides *which scripts run, in what order, and whether they need to run at all*. It reads `turbo.json` at the repo root, which defines a **task graph**: a description of which tasks depend on which other tasks, across every workspace package at once.

The critical line in our `turbo.json` is that `build`, `lint`, `check-types`, and `test` all declare `dependsOn: ["^build"]`. The caret means "the `build` task of every package this package depends on." In practice that means running `pnpm test` for `apps/web` doesn't just run `apps/web`'s own tests — Turborepo first walks the dependency graph, rebuilds `token-factory` and `peregrine-commit-ui` if their source has changed since the last build, and only then runs the tests, so `apps/web` is always tested against fresh, real `dist/` output rather than stale or raw source.

> Turborepo also **caches** task outputs. If nothing that could affect a package's build has changed, Turborepo replays the cached result instead of re-running the task — which is why CI stays fast even as the repo grows, and part of why CI caches the `.turbo` directory between runs.

The commands you'll actually run

Every command below is a thin wrapper defined in the root `package.json` that fans out via `turbo run <task>` to every workspace package that defines that script:

```
pnpm install            # one install for the whole workspace
pnpm dev                # long-running dev servers (not cached)
```

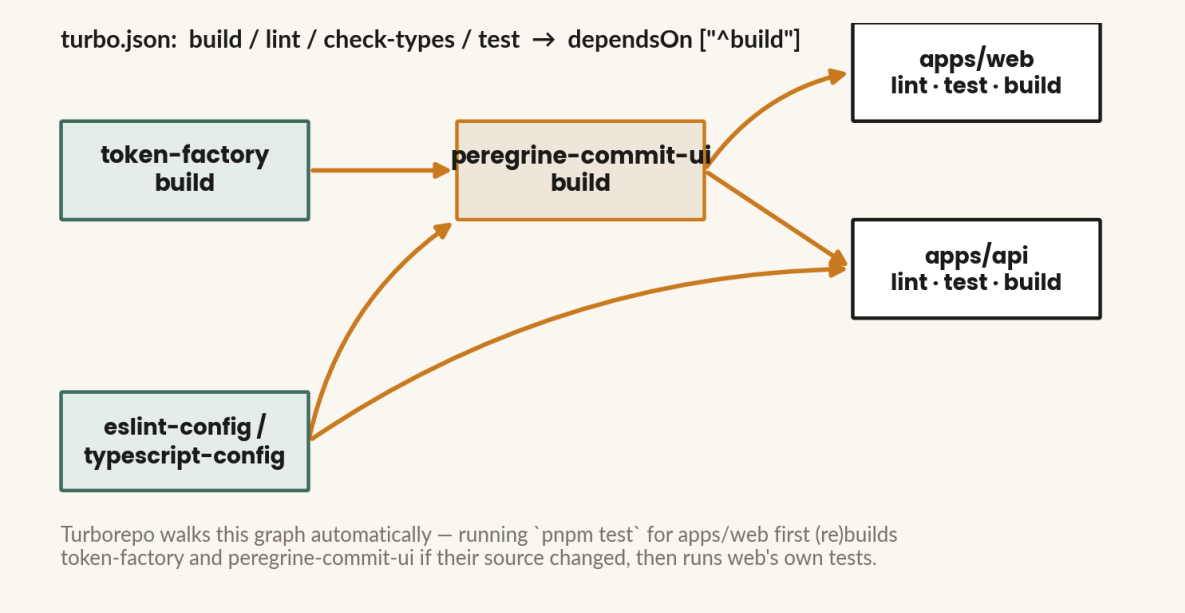

Figure 2: *Turborepo's task graph for this repo: build dependencies before testing or building what depends on them*

```
pnpm build              # turbo run build, dependency-ordered
pnpm lint               # eslint --max-warnings 0, per package
pnpm check-types        # tsc --noEmit, per package
pnpm test               # per-package test runner
pnpm format:check        # prettier --check . (not turbo'd — runs once, repo-wide)
pnpm storybook           # turbo run storybook, all packages that have one
```

Scope any of these to a single package with `--filter`, e.g. `pnpm --filter @peregrine-commit/ui test` — useful when you're iterating on one package and don't want to wait on the whole graph.

> **For designers**
> You will likely never type most of these commands directly. The one worth knowing is `pnpm storybook`, which starts a local server showing every component in isolation — the fastest way to browse what already exists before asking an engineer to build something new. See Chapter 10.

Chapter 3 --- Design Tokens: the Language of the Design System

What a design token is

A design token is a named, stored design decision — a color, a spacing value, a font size — kept as data instead of hard-coded inside components. The point of naming `#c97a1e` as a token rather than writing the hex code directly into a Button's CSS is that the *name* can be reused everywhere that color is needed, and the *value* behind that name can change in one place without hunting through every component that uses it.

Mature design token systems split tokens into two layers, and Peregrine Commit follows this pattern precisely.

Primitive tokens are raw, context-free values — the complete palette of everything the system could possibly use. A primitive doesn't know what it's *for*; `pc-rust-500` is just a hex code.

Semantic tokens are purpose-driven names that *reference* primitives. `color-accent` doesn't contain a hex code itself — it points at `pc-rust-500` — and it's the semantic name, not the primitive, that components actually use. This indirection is what makes theming possible: swap what `color-accent` points to, and every component using it updates automatically, with zero component code changed.

> **For designers**
> If you're used to Figma variables, this maps almost exactly onto Figma's own primitive-vs-semantic (or "raw" vs "alias") variable modes. A token named for what it *is* (`rust-500`) is a primitive; a token named for what it's *used for* (`color-accent`, `color-danger`) is semantic. When you hand off a design, referencing the semantic name in your notes ("this button uses `color-accent`") is far more durable than referencing a hex code.

Where our tokens actually live

The token system spans two packages, and it's important not to hand-edit the generated output of either:

`packages/token-factory` owns the actual values. `src/defaults/colors.ts`, `spacing.ts`, `typography.ts`, and `fonts.ts` each export a flat object of primitive tokens — for example, `colors.ts` defines primitives like:

```
// packages/token-factory/src/defaults/colors.ts
'pc-ink-900': '#1a1a18',
'pc-rust-500': '#c97a1e',
// ...and semantic aliases, in the same file:
'color-text-primary': 'var(--pc-ink-900)',
'color-accent': 'var(--pc-rust-500)',
'color-accent-hover': 'color-mix(in oklch, var(--pc-rust-500) 85%, black)',
```

Dark mode is defined right alongside light mode, in a second export (`defaultColorTokensDark`) that reuses the *exact same semantic key names* — `color-bg`, `color-accent`, and so on — but points each one at a different primitive. That reuse of names across light and dark is the entire foundation the dark-mode mechanism in Chapter 6 depends on.

`spacing.ts` defines the rhythm scale (`space-0` through `space-32`, running 2px to 128px), radii, border widths, shadows — including a shadow token that itself references a color token (`shadow-focus: 0 0 0 3px var(--color-focus-ring)`) — and motion tokens like `ease-standard` and `duration-*`. `typography.ts` defines font families and weights, plus composite tokens that bundle weight, size, line-height, and family into a single CSS `font` shorthand, e.g. `'text-body-md': 'var(--font-weight-400) 1rem/1.6 var(--font-body)'`.

A second layer lives in **`createSemanticTokens.ts`**: a pure function that takes primitive references in and derives *component-anatomy* tokens out — things like `inset-*/stack-*/inline-*` (spacing rhythm for insets, vertical stacks, and inline gaps), `control-height-*/control-pad-x/control-radius` (the anatomy of a form control, shared by Button, Input, Select, etc.), `surface-radius/surface-shadow-*` (the anatomy of a card-like surface), and `focus-ring/focus-ring-offset`. `defaults/semantic.ts` calls that factory with the canonical primitive references to produce the tokens the rest of the system consumes.

`mergeTokens(defaults, overrides)` is deliberately trivial — a shallow `{ ...defaults, ...overrides }` — but it's the escape hatch that makes future rebranding cheap: a client project can override just the handful of tokens that change and inherit everything else.

`packages/peregrine-commit-ui/src/theme/tokens/*.ts` is the consuming layer: one thin file per token type (`colors.ts`, `spacing.ts`, `typography.ts`, `fonts.ts`, `semantic.ts`), each of which imports the token-factory defaults, passes them through `mergeTokens` (currently a no-op — this package ships the stock brand, no overrides), and serializes the result into a CSS string via `tokensToCss`. `colors.ts` is the one file in this layer that does something extra — it emits *two* CSS blocks from the *same* function:

```
// packages/peregrine-commit-ui/src/theme/tokens/colors.ts
export const colorsCss =
  tokensToCss(mergeTokens(defaultColorTokensLight)) +
  tokensToCss(mergeTokens(defaultColorTokensDark), { selector: "[data-theme='dark']" });
```

`theme/tokens/base.ts` isn't tokens at all — it's a plain CSS reset (box-sizing, default background/-text color pulled from the color tokens, a focus-visible ring built from `shadow-focus`) that establishes sane defaults on raw HTML elements. `theme/tokens.css.ts` is the barrel that concatenates all of these CSS strings — fonts, colors, spacing, typography, semantic, base — into one `styled-components` `createGlobalStyle` block, which `PeregrineThemeProvider` mounts once at the root of the app.

Finally, `theme/theme.ts` builds a **typed mirror** of the same values as a plain JS object (`{ color: { accent: 'var(--color-accent)' }, space: { 6: 'var(--space-6)' }, ... }`) purely so that `props.theme.color.accent` autocompletes in your editor inside a styled-components template literal. It is documented as existing *only* for that convenience — `props.theme.color.accent` and hardcoding `var(--color-accent)` resolve to the identical string, because both ultimately point at the same CSS custom property. There is no second source of truth here.

Following one value end to end

The clearest way to internalize this system is to trace a single value all the way from its definition to a rendered pixel. Here's `color-accent`, the rust color used for primary buttons:

1. **Primitive**, in `token-factory/src/defaults/colors.ts`: `'pc-rust-500': '#c97a1e'`.

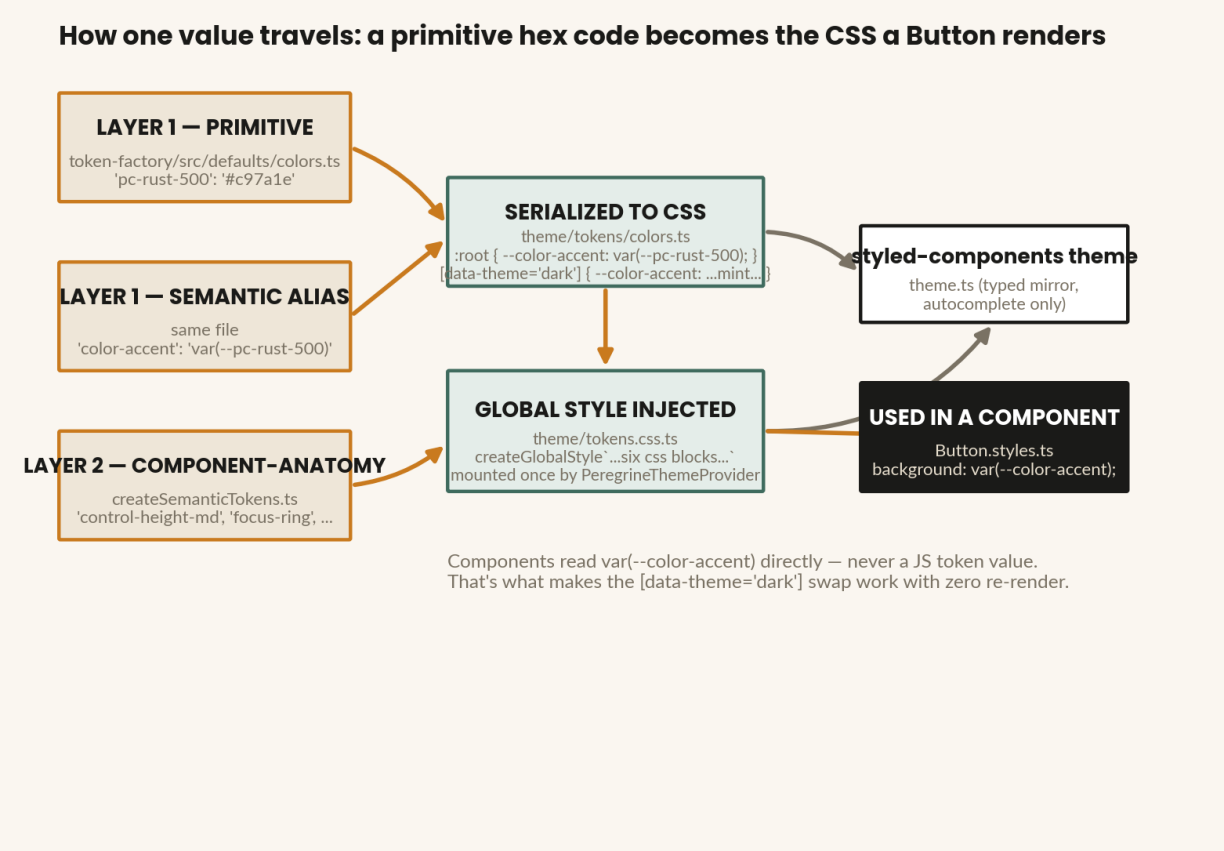

Figure 3: *One value's journey: a primitive hex code becomes the background of a Button*

2. **Semantic alias**, same file: `'color-accent': 'var(--pc-rust-500)'`.
3. **Serialized to CSS**, in `peregrine-commit-ui/theme/tokens/colors.ts`: written into `:root { --color-accent: var(--pc-rust-500); }`, and — under the dark palette — into `[data-theme='dark'] { --color-accent: <dark value>; }`.
4. **Injected once**, via `GlobalStyle` in `theme/tokens.css.ts`, mounted by `PeregrineThemeProvider`.
5. **Consumed directly**, in `atoms/Button/Button.styles.ts`: `background: var(--color-accent);` — not `props.theme.color.accent`, though it would resolve identically.

> Components read CSS custom properties (`var(--color-accent)`) directly. They never read a JavaScript token value at render time. This one decision is what makes dark mode a zero-JavaScript, zero-re-render CSS cascade — covered fully in Chapter 6.

> **For senior devs**
> There is currently no runtime rebrand prop on `PeregrineThemeProvider` — token composition is build-time only, resolved when `theme/tokens/*.ts` calls `mergeTokens()` with no overrides. If a future client engagement needs runtime brand-switching rather than compile-time, that's the seam to extend: `mergeTokens` already exists for exactly this purpose, it's just unused today.

Chapter 4 --- Atomic Design: Organizing the Component Library

The methodology, briefly

Atomic Design is a way of organizing UI components by increasing complexity, borrowed from chemistry's own hierarchy: atoms combine into molecules, molecules combine into organisms. Brad Frost formalized it as five tiers — atoms, molecules, organisms, templates, and pages — and its real value isn't the taxonomy itself but the **shared vocabulary** it gives a team. When a designer or engineer says "this should be an atom," everyone in the room understands roughly how small, how reusable, and how opinion-free that piece should be.

The rule that makes the hierarchy actually hold together in code, not just in theory, is that **each tier is composed only from the tier below it** — an atom never reaches sideways into another atom's internals, and a molecule never skips straight to raw CSS when an atom already exists for the job.

Our five tiers

Peregrine Commit adds one tier underneath Brad Frost's original three that we actively use: **primitives** — the truly irreducible layout building block beneath atoms. Above organisms, instead of generic "templates" and "pages," we have concrete **app sections**, since this repo currently serves one product (the marketing site) rather than a template library for arbitrary future pages.

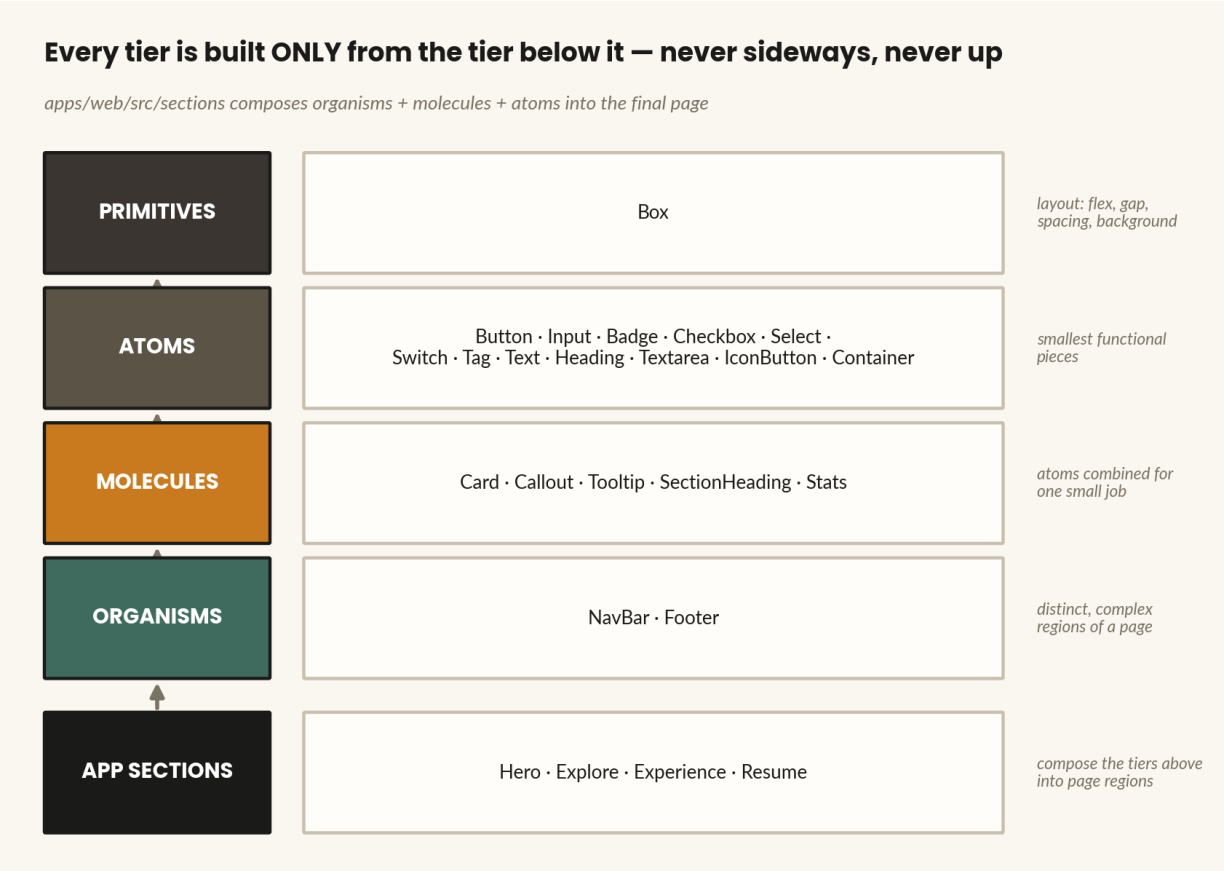

Figure 4: Every tier in peregrine-commit-ui is built only from the tier directly below it

Primitives (`packages/peregrine-commit-ui/src/primitives/`): today, just `Box` — the load-

bearing layout primitive that every other layout-oriented component should build on rather than styling a bare `div`. It owns flex/gap/alignment props, background variants (`bg`, `surface`, `surface-sunken`), and the shared spacing-prop mixin described below.

Atoms (`src/atoms/`, 12 today): `Badge`, `Button`, `Checkbox`, `Container`, `Heading`, `IconButton`, `Input`, `Select`, `Switch`, `Tag`, `Text`, `Textarea`. These are the smallest pieces that do something functional on their own.

Molecules (`src/molecules/`, 5): `Card`, `Callout`, `SectionHeading`, `Tooltip`, `Stats` — atoms combined to do one small, complete job.

Organisms (`src/organisms/`, 2): `NavBar`, `Footer` — distinct, complex regions built from molecules and atoms.

App sections (`apps/web/src/sections/`): `Hero`, `Explore`, `Experience`, `Resume` — these compose organisms, molecules, and atoms into the actual regions of the marketing page, and live in the consuming app, not the library.

> **For junior devs**
> Before building anything new, walk one tier down. Need a new molecule? Check whether the atoms it needs already exist. Need a new atom? Check whether `Box` already gives you the layout you need and you're really just adding variant styling. Almost everything in this repo is composition, not invention.

The file shape every component follows, with no exceptions

Every primitive, atom, molecule, and organism in `peregrine-commit-ui` lives in its own directory with exactly five files:

```
Button/
├── Button.tsx          # public prop API + logic
├── Button.styles.ts     # styled-components definitions
├── Button.test.tsx      # Vitest + Testing Library + jest-axe
├── Button.stories.tsx   # Storybook CSF3 stories
└── index.ts             # barrel re-export
```

When you're building something new, find the closest existing analog and copy its shape rather than inventing a new pattern. `Container` is the simplest real example of pure composition in the repo — it's `Box` with three lines of extra CSS:

```tsx
// atoms/Container/Container.tsx
export type ContainerProps = BoxProps;
export function Container(props: ContainerProps) {
  return <StyledContainer {...props} />;
}
```

```ts
// atoms/Container/Container.styles.ts
export const StyledContainer = styled(Box)`
  max-width: var(--container-max);
  margin-left: auto;
```

```
  margin-right: auto;
  padding-left: var(--container-pad);
  padding-right: var(--container-pad);
`;
```

Every public export — every component and its prop/variant types — is re-exported from the top-level `packages/peregrine-commit-ui/src/index.ts`, grouped by tier under comment headers (`// Primitives`, `// Atoms`, `// Molecules`, `// Organisms`). There's no automatic folder-level barrel aggregation beneath that: nothing outside the package can import a component unless it appears in that one file.

> If you add a component but forget to add it to `src/index.ts`, it will build fine, test fine, and show up in Storybook — but no other package can import it. This is the single most common "why isn't my new component available in `apps/web`" gotcha.

Chapter 5 --- Styling: styled-components and Transient Props

The problem transient props solve

`styled-components` lets you attach arbitrary props to a styled element and read them inside its CSS template literal to branch styling logic — but by default, any prop you pass also gets forwarded to the underlying DOM node. Pass `variant="primary"` to a `styled.button`, and React dutifully renders `<button variant="primary">`, which is not a real HTML attribute and produces a console warning (and invalid markup).

The fix `styled-components` provides is the **`$` prefix**: any prop name starting with `$` is treated as "transient" — available inside the styled component's CSS, but automatically stripped before anything reaches the DOM.

How the pattern is used here

Every styled component in `peregrine-commit-ui` follows the same split: the public-facing `.tsx` file exposes a clean, semantic prop API (`variant`, `size`, plain booleans), and translates each prop into its `$`-prefixed transient equivalent right before handing off to the styled component. Button is the canonical example:

```
// atoms/Button/Button.tsx
export function Button({
  variant = 'primary', size = 'md', type = 'button', ...props
}: ButtonProps) {
  const [spacing, rest] = extractSpacingProps(props);
  return (
    <StyledButton $variant={variant} $size={size} $spacing={spacing}
                  type={type} {...rest} />
  );
}
```

```
// atoms/Button/Button.styles.ts
export const StyledButton = styled.button<{
  $variant: ButtonVariant; $size: ButtonSize; $spacing: SpacingProps;
}>`
  /* shared base styles */
  ${({ $size }) => sizeStyles[$size]}
  ${({ $variant }) => variantStyles[$variant]}
  ${spacingCss}
`;
```

`sizeStyles` and `variantStyles` are lookup tables typed as `Record<ButtonSize, css...>` / `Record<ButtonVariant, css...>` — so adding a new size or variant is a matter of adding one new key to a table, not writing new conditional logic.

The shared spacing-prop mixin

One more pattern threads through nearly every component: `SpacingProps` (defined in `theme/spacingProps.ts`) lets *any* component accept layout props like `p`, `px`, `mt`, typed against the

token-factory `SpaceKey` union so only real spacing-token values (`space-4`, `space-8`, etc.) type-check. `extractSpacingProps` strips those props out of what reaches the DOM, and `spacingCss` is a shared template literal that turns whichever spacing props were passed into real CSS declarations. This is why you'll see `[spacing, rest] = extractSpacingProps(props)` at the top of nearly every component's function body — it's the one-line idiom for "let this component also accept spacing overrides."

For senior devs

Because `SpacingProps` values are typed as template-literal types derived directly from token-factory's `SpaceKey` union (`` `space-${SpaceKey}` ``), adding a new spacing token to `token-factory/defaults/spacing.ts` automatically makes it a valid value everywhere `p`/`mt`/`px` etc. are accepted, with no separate type definition to update.

Chapter 6 --- Light and Dark Mode

The mechanism, precisely

Because every color a component uses is a CSS custom property (`var(--color-accent)`, never a hardcoded value or a JS token lookup), theming reduces to a CSS problem, not a React problem. `theme/tokens/colors.ts` writes the light palette to `:root` and the *same variable names*, under dark values, into a `[data-theme='dark'] { ... }` block. Because CSS custom properties cascade down through the DOM, any element inside a subtree with `data-theme="dark"` on an ancestor picks up the dark values automatically — no component re-render required.

`PeregrineThemeProvider` wires this to a `mode` prop:

```
export function PeregrineThemeProvider({ children, mode = 'light' }) {
  return (
    <StyledThemeProvider theme={theme}>
      <GlobalStyle />
      <div data-theme={mode === 'dark' ? 'dark' : undefined}>
        {children}
      </div>
    </StyledThemeProvider>
  );
}
```

`apps/web/src/hooks/useThemeMode.ts` owns the actual mode state: it seeds from `localStorage`, falls back to the `prefers-color-scheme` media query if nothing is stored, persists on every change, and exposes `{ mode, setMode, toggleMode }`. `AppLayout` calls that hook and feeds `mode` into `PeregrineThemeProvider`, and passes `toggleMode` down to the `Navbar`'s theme toggle button.

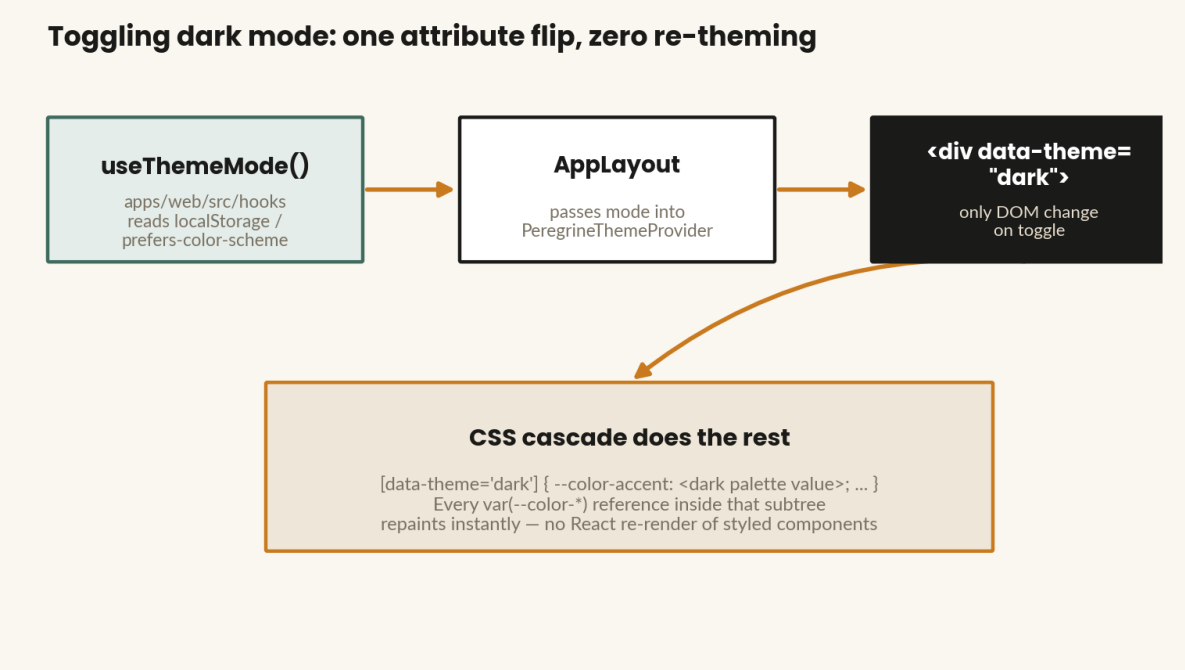

Figure 5: Toggling dark mode is one DOM attribute flip; the CSS cascade repaints everything

> Only color tokens have a dark variant. Spacing, typography, and semantic (component-anatomy) tokens are the same in both modes — dark mode in this system is purely a repaint, never a re-layout.

Storybook mirrors the exact same mechanism for previewing components in isolation. Its `.storybook/preview.tsx` config adds a `theme` toolbar global, and a decorator passes its value into `PeregrineThemeProvider` around every story.

Chapter 7 --- apps/web: the Marketing Site

`apps/web` is a React 18 + TypeScript + Vite application, and deliberately contains almost no styling of its own — every visual decision is delegated to `@peregrine-commit/ui`.

The composition is intentionally linear and easy to reorder. `src/App.tsx` renders `AppLayout` wrapping an ordered list of section components imported from `src/sections/`: `Hero`, `Explore`, `Experience`, `Resume`. Reordering the page means reordering that list — not touching the sections themselves. Each section is a standalone component built purely from `@peregrine-commit/ui` primitives, atoms, and molecules; for example `Hero.tsx` composes `Box`, `Button`, `Container`, `SectionHeading`, and `Stats` with no bespoke CSS.

`AppLayout` (`src/layouts/AppLayout`) is the one place app-level concerns live: it calls `useThemeMode()`, wraps everything in `PeregrineThemeProvider`, and renders `Navbar` above and `Footer` below the page content.

Testing and Storybook

`apps/web` tests with **Jest + React Testing Library** (`jest.config.js`: `ts-jest` preset, `testEnvironment: 'jsdom'`, `jest-dom` matchers loaded via a setup file) — deliberately different tooling from `peregrine-commit-ui`'s Vitest setup, discussed in Chapter 9. Its Storybook instance runs on **port 6007** (`storybook dev -p 6007`), configured with `@storybook/addon-essentials` and `@storybook/addon-a11y`; today it holds a single story, `App.stories.tsx`, showing the whole composed page rather than isolated components — useful as a reference for how sections assemble together.

Chapter 8 --- apps/api: the Backend Service

`apps/api` is a small, deliberately minimal **Express 5 + TypeScript** service — plain Node, not a framework like NestJS.

`src/app.ts` exports a `createApp()` factory rather than a singleton app instance, which is what makes the service trivially testable: each test gets a fresh app. The middleware chain is `express.json()` → a rate limiter → the health router → a 404 catch-all → an error handler. `src/server.ts` just calls `createApp().listen(PORT)`.

Rate limiting

`src/middleware/rateLimiter.ts` wraps `express-rate-limit`'s `rateLimit()`, configured from environment variables (`RATE_LIMIT_WINDOW_MS`, default 15 minutes; `RATE_LIMIT_MAX`, default 100 requests), with standard `RateLimit-*` headers enabled and legacy headers disabled.

Testing

`src/app.test.ts` uses **Jest + Supertest**: it calls `createApp()` fresh per test and asserts `/health` returns `200 { status: 'ok' }` and unknown routes return `404`. A dedicated `describe('rate limiting')` block sets `RATE_LIMIT_MAX=2`, calls `jest.resetModules()`, and dynamically re-imports the app to get a fresh limiter instance, then proves the third request in the window returns `429`. Unlike `apps/web`'s Jest config, `apps/api`'s is deliberately simple — `testEnvironment: 'node'`, no `jsdom`, no setup files — because an HTTP API test never touches a DOM.

Database

`docker/docker-compose.yml` defines a single `mysql:8.4` service, fully environment-driven with sane local defaults, health-checked via `mysqladmin ping`, and persisted to a named volume. `apps/api/.env.example` documents the API's own runtime environment variables, including database connection settings.

> **For senior devs**
> As of this writing, `apps/api` doesn't yet actually connect to MySQL anywhere in source — there's no DB client or ORM code, only the env-var scaffolding and the Compose service. If you're the one wiring up the first real database call, that's greenfield, not a bug you're missing.

Chapter 9 --- Quality Tooling

ESLint: one shared flat config

`packages/eslint-config` exports a single shared **flat config** (the modern ESLint config format, an array of config objects rather than the older `.eslintrc` cascading format), built with `typescript-eslint`'s `tseslint.config(...)` helper. Every app and package has its own one-line `eslint.config.mjs` that imports and re-exports this shared config, so there is exactly one place linting rules are actually defined.

The layers, in order: ignore build output (`dist/`, `storybook-static/`, `coverage/`) → ESLint's own recommended rules → `typescript-eslint`'s **strict** and **stylistic** rule sets (a stricter bar than just "recommended") → React, `react-hooks`, and `jsx-a11y` plugin rules, including the JSX runtime rule that means you never need `import React` → a couple of house rules (interfaces over type aliases for object shapes; unused variables prefixed with _ are allowed) → Storybook's recommended rules → `eslint-config-prettier` last, which turns off any stylistic rule that would conflict with Prettier's formatting. CI runs this with `--max-warnings 0`, so warnings fail the build exactly like errors.

TypeScript: four configs, four runtime targets

`packages/typescript-config` exports exactly four base `tsconfig.json` files, and which one a package extends tells you what kind of package it is:

- **`base.json`** — the shared root: ES2022 target, strict mode fully on, `noUnusedLocals`/`noUnusedParameters`, `verbatimModuleSyntax`. Never used standalone.
- **`node-library.json`** — extends base for CommonJS Node output (`module: CommonJS`, `moduleResolution: Node10`). Used by `apps/api`, a plain Node service.
- **`react-app.json`** — extends base with DOM libs and `noEmit: true` (Vite handles the actual compilation). Used by `apps/web`.
- **`react-library.json`** — extends base with DOM libs plus `declaration`/`declarationMap: true`, since this package ships `.d.ts` files for consumers. Used by `peregrine-commit-ui`.

`token-factory` is the interesting exception: it extends `base.json` directly, not `react-library.json`, because it's a plain-TypeScript library with no React or JSX in it at all — its Storybook even uses `@storybook/html`, not `@storybook/react`, since it's rendering token swatches as plain DOM elements, not components.

Testing: Vitest here, Jest there --- and why

`peregrine-commit-ui` and `token-factory` both use **Vitest**; `apps/web` and `apps/api` both use **Jest**. This isn't inconsistency, it's a deliberate split along a real seam: Vitest is Vite-native — it shares Vite's module graph and uses `esbuild` for TypeScript, which makes it dramatically faster in watch mode and requires effectively no extra config in a Vite-based package. It's the natural choice for the two *library* packages, which are built with Vite-family tooling already. Jest predates Vitest by years and remains the more battle-tested choice for full applications with more complex test-environment needs (`apps/web`'s tests run against `jsdom` for component rendering; `apps/api`'s run against plain `node` for HTTP-level testing) — and its API is close enough to Vitest's (`describe`/`it`/`expect`, `vi.fn()` vs `jest.fn()`) that moving between the two codebases doesn't require relearning a testing mental

model, just swapping a couple of import names.

`peregrine-commit-ui`'s Vitest setup adds one more layer worth knowing about: **`jest-axe`**, wired in via `vitest.setup.ts` (`expect.extend` with `toHaveNoViolations`). Nearly every component test includes an explicit accessibility assertion:

```
expect(await axe(container)).toHaveNoViolations();
```

> **For junior devs**
> That one line is not boilerplate to skip past — it's a real automated accessibility check (contrast, ARIA attributes, semantic markup) run on every component, every test run. If you add a new component and its test fails on that line, treat it exactly as seriously as a failing assertion on the component's actual behavior.

Chapter 10 --- Storybook & Component-Driven Development

Why build components in isolation

Component-Driven Development is the practice of building and verifying a UI component on its own, outside the full application, before wiring it into a real page. The advantage is that a component built in isolation is forced to declare all of its own dependencies (props, not implicit page context), which tends to produce more reusable, more testable components — and it gives designers and product stakeholders something concrete to review long before a feature is fully assembled.

Storybook is the tool that makes this practical: it's a standalone dev server that renders one component at a time against different prop combinations ("stories"), with live controls to tweak props in the browser. Modern Storybook stories are written in **CSF3** (Component Story Format 3) — each file default-exports metadata about the component, and named-exports one object per state you want to showcase, with far less boilerplate than earlier formats:

```
// atoms/Button/Button.stories.tsx (shape, not exact code)
export default {
  component: Button,
  argTypes: { variant: { control: 'select' }, size: { control: 'select' } },
};

export const Primary = { args: { variant: 'primary' } };
export const Disabled = { args: { disabled: true } };
```

Where to look in this repo

There are **three separate Storybook instances**, one per package that has UI to show, each on its own port so you can run more than one at a time:

- `packages/peregrine-commit-ui` → **port 6006** — the component library itself; every atom, molecule, and organism has stories here.
- `apps/web` → **port 6007** — currently just `App.stories.tsx`, a whole-page reference.
- `packages/token-factory` → **port 6010**, using `@storybook/html` (not React) to render raw token swatches.

A few specific stories are worth opening first: `atoms/Button/Button.stories.tsx` is the textbook example of controls plus one named story per variant. `molecules/Card/Card.stories.tsx` shows the `fn()` mock pattern from `@storybook/test` for interaction props like `onClick`. `organisms/NavBar/NavBar.stories.tsx` shows `parameters: { layout: 'fullscreen' }`, the setting you need for any organism too wide for Storybook's default padded canvas.

For designers

`packages/token-factory/src/stories/Colors.stories.ts` (and its siblings `Spacing`, `Typography`, `Radius`, `Shadows`, `Semantic`) render every primitive and semantic token as a literal swatch, light and dark side by side, with no component involved — the single fastest way to see the entire palette in one place and check a hex value against what's actually shipped, rather than trusting a Figma file might be in sync.

Chapter 11 --- Continuous Integration

`.github/workflows/ci.yml` runs one job on every push and pull request targeting `main`, with a concurrency group that cancels superseded runs so an outdated push doesn't waste CI time. The steps run in a fixed, linear order:

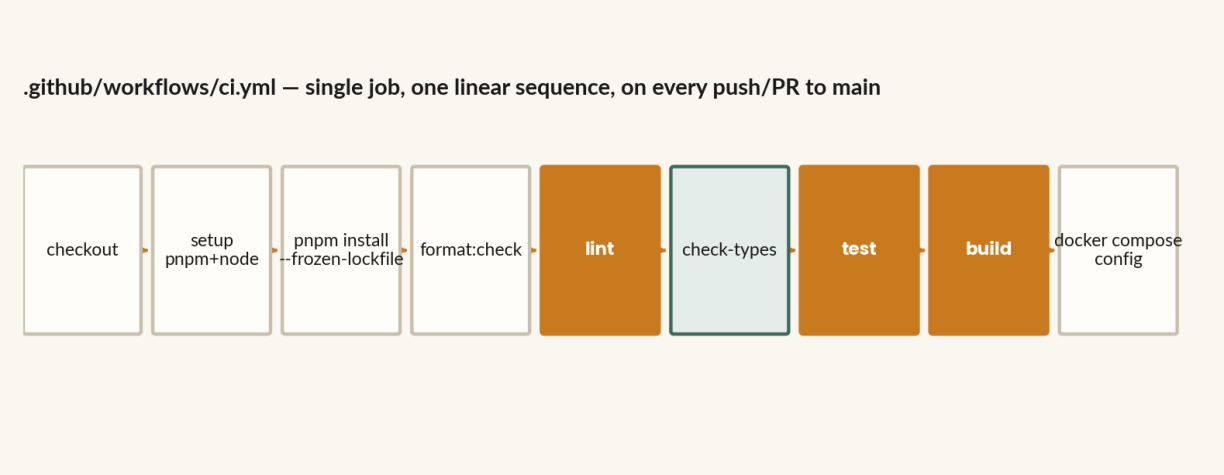

Figure 6: CI runs the exact same quality gates you should run locally, in the same order, before every push

1. Check out the repository.
2. Set up pnpm (version resolved from the `packageManager` field in root `package.json`) and Node (version from `.nvmrc`), with pnpm's store cached.
3. `pnpm install --frozen-lockfile` — fails the build outright if the lockfile is out of sync with any `package.json`, rather than silently updating it.
4. Restore a cache of the `.turbo` directory, keyed on the commit SHA.
5. `pnpm format:check` — Prettier, checking only.
6. `pnpm lint` — `turbo run lint` across every package.
7. `pnpm check-types` — `turbo run check-types` across every package.
8. `pnpm test` — `turbo run test` across every package.
9. `pnpm build` — `turbo run build` across every package.
10. `docker compose -f docker/docker-compose.yml config` — validates that the Compose file parses correctly; it does not actually start MySQL or run integration tests against it.

> This is the exact order the root `CLAUDE.md` tells you to run locally before pushing: `format:check` → `lint` → `check-types` → `test` → `build`. Matching that order locally means you'll never be surprised by CI — if something's going to fail, it fails on your machine first, faster than waiting on a CI run.

Chapter 12 --- Cheat Sheet

Commands

```
pnpm install                                  # install everything
pnpm dev                                      # run dev servers
pnpm build / lint / check-types / test        # repo-wide, dependency-ordered
pnpm format:check                             # Prettier check (not turbo'd)
pnpm storybook                                # all Storybooks that exist
pnpm --filter @peregrine-commit/ui test       # scope any task to one package
pnpm --filter @peregrine-commit/ui storybook  # ui's Storybook only (port 6006)
pnpm docker:up / pnpm docker:down             # local MySQL for apps/api work
```

Where things live

- Design token **values** → `packages/token-factory/src/defaults/*.ts`
- Design token **derivation logic** → `packages/token-factory/src/createSemanticTokens.ts`
- Token → CSS **serialization** → `packages/peregrine-commit-ui/src/theme/tokens/*.ts`
- **Components** → `packages/peregrine-commit-ui/src/` — `primitives/`, `atoms/`, `molecules/`, `organisms/`
- **Public exports** → `packages/peregrine-commit-ui/src/index.ts` (nothing is importable elsewhere without this)
- **Page composition** → `apps/web/src/App.tsx` and `apps/web/src/sections/`
- **Theme mode state** → `apps/web/src/hooks/useThemeMode.ts`
- **Shared lint rules** → `packages/eslint-config/index.js`
- **Shared tsconfig bases** → `packages/typescript-config/*.json`
- **CI definition** → `.github/workflows/ci.yml`

Storybook ports

Package	Port	Framework
`peregrine-commit-ui`	6006	`@storybook/react-vite`
`apps/web`	6007	`@storybook/react-vite`
`token-factory`	6010	`@storybook/html-vite`

Chapter 13 --- Where to Start, By Role

If you're a junior developer

Start by running `pnpm storybook` and clicking through every component in `peregrine-commit-ui`. Then open `atoms/Button/` end to end — all five files — and read it as a template. Your first few tasks will almost always be one of: adding a new variant to an existing component (edit the lookup table in `X.styles.ts`), building a new molecule from existing atoms, or wiring an existing component into a new spot in an `apps/web` section. Before writing new CSS, check whether `Box` and the shared `SpacingProps` mixin already give you what you need.

If you're a senior developer

The seams worth understanding deeply before you touch them are: the `turbo.json` task graph and its `^build` dependencies (Chapter 2), the `mergeTokens` escape hatch that has no current caller but is clearly meant for future rebranding (Chapter 3), and the deliberate Vitest/Jest split (Chapter 9) — all three represent real architectural decisions, not accidents, and CLAUDE.md is explicit that new code should extend these patterns rather than introduce parallel ones.

If you're a product designer

Run `pnpm storybook` and start with `token-factory`'s Storybook on port 6010 — the `Colors`, `Spacing`, and `Typography` stories are the ground truth for the palette and scale, more reliable than any static reference doc. When you hand off a design, referencing semantic token names (`color-accent`, `space-6`) rather than raw hex codes or pixel values lets engineers implement it as a token reference rather than a magic number, which is what keeps the system consistent as it grows. If a design calls for something that doesn't map to an existing atom or molecule, that's a useful signal worth raising directly with engineering — it may mean either an existing component needs a new variant, or the design itself should reconsider using what's already available.

Appendix --- Further Reading

This guide covers Peregrine Commit's specific implementation. For deeper background on the general concepts it builds on:

- Turborepo's own documentation on structuring a repository: https://turborepo.dev/docs/crafting-your-repository/structuring-a-repository
- Brad Frost's original Atomic Design methodology: https://atomicdesign.bradfrost.com/chapter-2/
- Storybook's Component Story Format reference: https://storybook.js.org/docs/api/csf
- styled-components' own docs on transient props: https://styled-components.com/docs/api